GEOG 761 Lab 5

Answer Proforma

Q1: With a tile_size of 512, what is the area in m^2 covered by a single tile?

(1 pt)

A tile is 512 by 512 pixels and Sentinel 2 optical bands are delivered at 10 m ground sampling, so a tile spans 5120 m on each side. The area is therefore 5120 times 5120, which is 26 214 400 square metres, roughly 26.2 square kilometres.

Q2: Explain why using U-Net and ResNet34 together is a good choice when working with multi-spectral imagery.

(5 pts)

The two components solve different halves of the problem. U-Net is an encoder decoder with skip connections: the encoder compresses the scene into progressively more abstract features while the decoder upsamples back to full resolution, and the skip connections copy the fine spatial detail from early layers straight across to the decoder, which is what lets the output trace water boundaries at pixel precision instead of producing blobs. ResNet34 as the encoder brings two things: residual connections that keep gradients healthy so a 34 layer network trains quickly and stably on a modest dataset, and ImageNet pretrained weights, so the early layers already detect edges and textures and we only fine tune rather than learn vision from scratch. For multispectral imagery the combination is particularly apt: the first convolution is rebuilt to accept six channels, so the network can learn spectral combinations such as the strong absorption of water in the near infrared, the deep encoder mixes those spectral cues with spatial context, and the skip connections make sure the spectrally derived classification still lands on crisp geographic boundaries. It is also a pragmatic middle weight: heavier encoders would overfit these few thousand tiles and train far slower for little gain.

Q3: What is the learning_rate and what is the likely effect that reducing it will have on your model?

(4 pts)

The learning rate is 0.001, the step size scaling factor that the Adam optimiser applies to every weight update. Reducing it makes each update smaller, so training becomes more cautious: the loss curve descends more smoothly with less oscillation, and the optimum found is often slightly finer because the optimiser does not overshoot narrow minima. The price is speed, since with a fixed budget of a few epochs a much lower rate can leave the model undertrained and the validation IoU visibly worse. In practice one lowers the learning rate when the loss bounces around or the validation metrics zig zag, and keeps it, or schedules a decay, when the curves are already smooth.

Q4: Increase the number of training epochs to 10 (this will take some time, so maybe come back to this question later if need be). Present the resulting graphs of performance metrics in a figure and interpret the results for a technical audience. As always, figures should be of publication quality.

(10 pts)

The ten epoch run reaches its best validation IoU of 0.893 at epoch 7 and finishes at 0.889, against 0.866 for the three epoch baseline, with validation loss bottoming near 0.066 around epoch 8. Nearly all of the gain arrives in the first four epochs, after which the curves plateau and oscillate within about three IoU points, an effect of the modest validation split and a constant learning rate of 0.001 with no decay schedule. Training loss sits almost flat near 0.26 from epoch 2 onward while validation loss keeps drifting down, so there is no overfitting signature; the optimiser is wandering around a shallow optimum. Precision holds between 0.94 and 0.96 throughout while recall climbs from about 0.85 to 0.93, so the additional epochs mostly teach the model to recover more of the true water rather than to stop inventing water. The practical reading is that ten epochs harvests the available easy gains on this dataset, and further progress would come from a learning rate schedule, augmentation or more scenes rather than from more epochs alone.

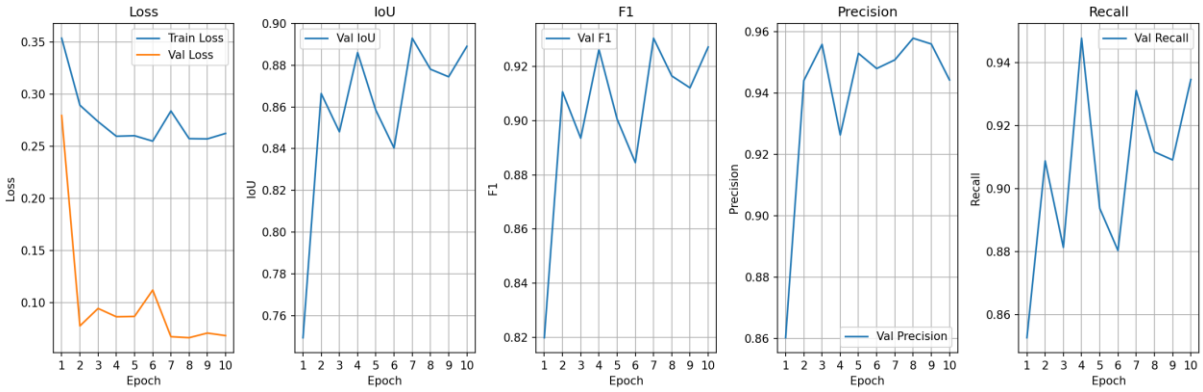


Figure: training and validation metrics of the water segmentation model retrained for ten epochs with everything else unchanged.

Q5: What is this vectorize operation doing to the raster mask output by the model and why are we doing so?

(5 pts)

Orthogonalize turns the raster mask, a grid of per pixel class values, into vector polygons and then regularises their outlines: nearly straight runs of pixel edges are simplified with an epsilon tolerance of 2, corners are squared, tiny fragments below the minimum area are discarded, and the result is a set of clean right angled polygons. We do this because solar panels are rectilinear human made objects, so pixel stair stepping along their edges is noise rather than information, and because most of the things one does downstream, counting installations, measuring their area, joining to cadastral or asset databases, editing in a GIS, need discrete georeferenced

objects with attributes rather than a field of pixels. Vectorisation is thus both a cleanup, enforcing the prior that the target objects have straight edges, and a change of data model into the form the application actually consumes.

Q6: Discuss the performance of the prediction in our example given above. Do so in reference to the ability of the prediction to resolve key features as compared to the ground truth (e.g. the river vs. the flood ponds). Explain why perfomance is varying in terms of both satellite data physical principles and the limitations of model and its training.

(10 pts)

The prediction and the truth agree almost perfectly on the large water bodies: the lakes and the bigger flood ponds are recovered with clean boundaries, which is why the headline IoU looks strong. The failures concentrate on exactly the features one would expect from the physics. The meandering river that runs the length of the scene is continuous in the ground truth but comes back only as scattered fragments in the prediction, because at 10 m ground sampling a channel one to three pixels wide is a mixed pixel problem: each river pixel blends water with bank soil and riparian vegetation, so its spectrum sits between the classes and the network, trained mostly on pure water pixels from lakes, drops it. The same applies to the many small ponds that appear in the truth but not in the prediction, objects at or below the resolution limit whose spectra never reach pure water. Deep clear water is trivially separable because it is dark across the near infrared bands, while shallow, turbid or vegetated water drifts spectrally towards soil and canopy and gets missed. On the model side the limitations compound the physics: only a few epochs of training with a plain cross entropy loss, which is dominated by the large easy water bodies and pays almost nothing for dropping a thin line of river pixels, and an encoder decoder architecture that must reconstruct one pixel structures through repeated downsampling. A boundary weighted or IoU style loss, more epochs and training tiles centred on channels would recover more of the river.

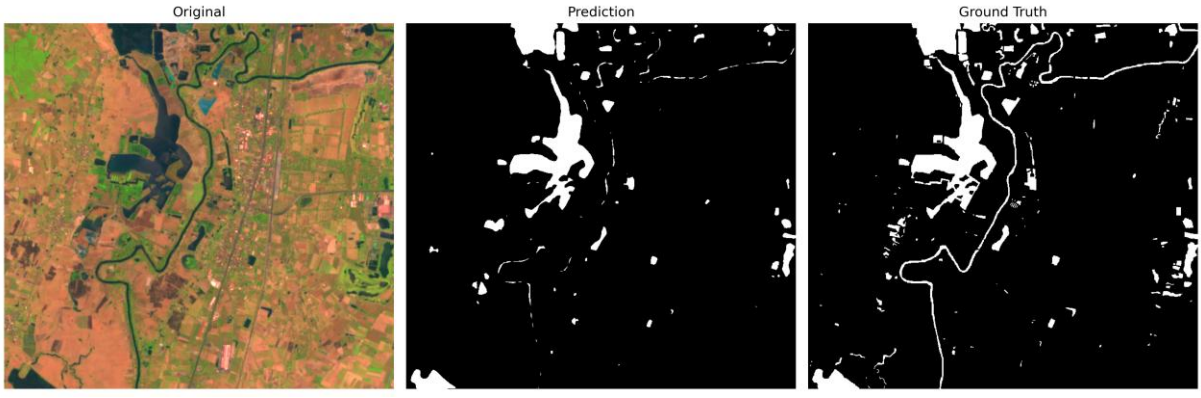


Figure: validation scene comparison, original six band composite, model prediction and ground truth for scene S2A L2A 20190318.

Q7 Exercise: using the code given above, the linked materials and any other sources you can find, improve the output of the model in order to better segment solar panels in the given data. In your answer:

GEOG 761 Lab 5

- Produce a publication quality figure of the resulting output.
- Write a paragraph that describes the steps you took and why.

Make sure to reference any materials used.

(15 pts)

I made three changes. First, the training tiles were regenerated with the stride halved from 256 to 128, which roughly quadruples the number of tiles the model sees and presents every solar array in many more positions. Second, training was extended from 10 to 30 epochs with early stopping on validation loss and patience of 8, letting the model keep learning while insuring against a wasted tail. Third, the vectorisation was tightened by raising the minimum polygon area from 10 to 40 square metres so that pixel scale speckle never reaches the output. The effect is large: best validation IoU rises from 0.871 to 0.954 at epoch 23, with precision 0.989 and recall 0.964, and on the unseen test raster the vectorised output grows from 41 to 55 polygons. The figure shows where the extra detections come from: the improved model recovers the large multi module array on the brown roof near the top of the scene and several arrays on the eastern block that the baseline missed entirely, and its outlines follow the module e dges more tightly. It is not perfect: one or two small false detections appear on the park courts, and a very low contrast array on a dark roof is still missed, both of which point to training data diversity rather than optimisation as the next lever. Materials used: the GeoAI documentation at opengeoai.org for train segmentation model, semantic segmentation and orthogonalize parameters, the segmentation models pytorch documentation for the U-Net and ResNet34 encoder, and the lab notebook itself.



Figure: predicted solar panel polygons over the unseen test raster, baseline model on the left and the improved training and post processing on the right.